

Experiential Learning Project Assessment-I

on

"Online Polling & Voting System"

under the course

Python Programming Language (N-PCCCM304P)

**Presented By
Sanay N Raut
CM25111**

**Under the Guidance of
Mrs. Mayuri Getme**

**Session: 2026-27 (ODD)
Department of CSE (AI & ML)**

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR

An Autonomous Institute, Affiliated to RTMNU, Nagpur

Contents

- 1 Problem Statement
- 2 Objectives
- 3 Algorithm
- 4 Flowchart / Pseudocode
- 5 Python Logic
- 6 Working & Graphical Representation
- 7 GitHub & Individual Contribution
- 8 Result / Expected Output

Problem Statement

Manual Systems Are Slow

Conducting polls manually is slow and prone to human error.

Duplicate Voting Risk

Manual systems struggle to prevent voters from casting more than one vote.

Tedious Vote Counting

Counting votes by hand for large numbers of participants becomes error-prone.

Need for Real-Time Stats

Candidates and voters need a simple way to view results and statistics as voting progresses.

Python Solution: arrays/lists store candidate names and vote counts, record votes, tally results, display real-time statistics, and validate voter identity automatically.

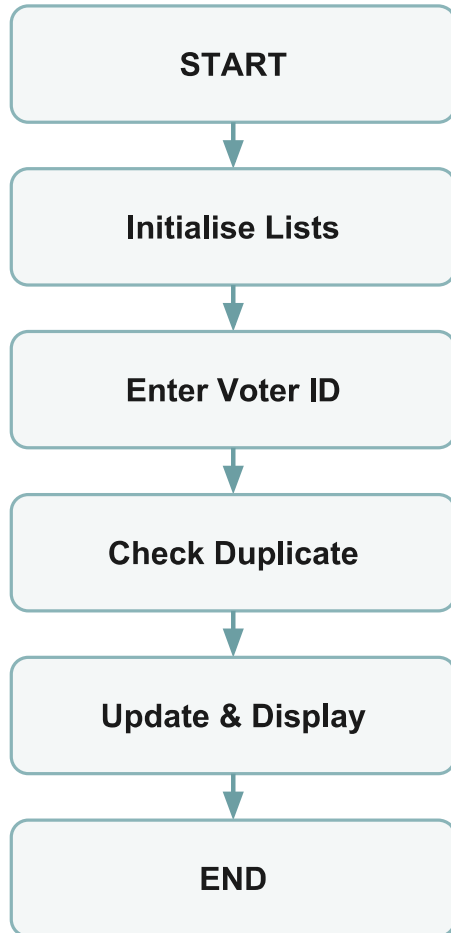
Objectives

- ✓ Develop an online polling system using Python arrays/lists for efficient storage of candidate names and vote counts.
- ✓ Allow voters to cast a vote for a candidate of their choice.
- ✓ Track voters who have already voted to prevent duplicate voting.
- ✓ Count and tally votes accurately for every candidate.
- ✓ Display real-time statistics including leading candidate and vote percentages.

Algorithm

- 1 Initialise candidate list, vote count array (zero), and empty voted-IDs list.
- 2 Enter voter ID and candidate choice.
- 3 Check for duplicates: if voter ID exists in voted list, reject with "Duplicate Vote Not Allowed".
- 4 Record valid vote: increment vote count for selected candidate; add voter ID to voted list.
- 5 Display statistics: show total votes, per-candidate counts, and leading candidate.
- 6 Repeat for next voter until voting ends, then stop.

Flowchart / Pseudocode



PSEUDOCODE

```
root <- Node("Root")
candidates = ["A","B","C"]
votes = [0,0,0]
voted_ids = []

WHILE voting_open:
    read voter_id, choice
    IF voter_id in voted_ids:
        print "Duplicate Vote
            Not Allowed"
    ELSE:
        votes[choice] += 1
        voted_ids.append(voter_id)
        print "Vote Recorded"
        display statistics
END WHILE
```

Python Logic

```
candidates = ["A", "B", "C"]
votes = [0, 0, 0]
voted_ids = []

voter_id = "V1001"
choice = 0

if voter_id in voted_ids:
    print("Duplicate Vote Not Allowed")
else:
    votes[choice] = votes[choice] + 1
    voted_ids.append(voter_id)
    print("Vote Recorded")

print("A:", votes[0])
print("B:", votes[1])
print("C:", votes[2])
```

Arrays / Lists

Store candidate names and vote counts using indexed data structures.

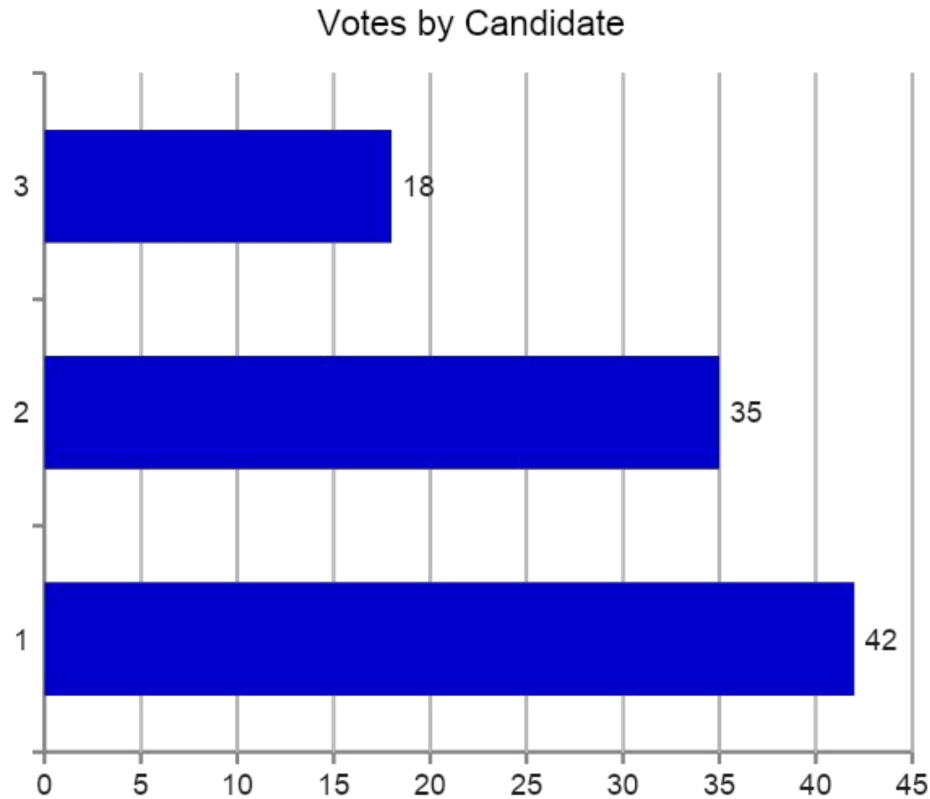
Duplicate Check

if voter_id in voted_ids prevents repeat voting.

Percentage Calc

$(\text{votes}[i] / \text{total} * 100)$ gives live vote share.

Working & Graphical Representation



Cast Vote

Voter provides unique ID; vote count array incremented at candidate's index if new.

Prevent Duplicates

Repeated voter ID rejected with "Duplicate Vote Not Allowed" message.

Real-Time Stats

Total votes, per-candidate counts, percentages, and leading candidate displayed after every valid vote.

Efficiency

$O(1)$ vote update, $O(n)$ result tally — efficient for any number of candidates.

GitHub & Individual Contribution

GitHub Setup

Created account, repository "Online Polling & Voting System", uploaded Python files with meaningful commits. Used branches / pull requests for collaboration and tracked all project updates.

My Contribution

Problem analysis, algorithm design, Python implementation (vote casting, duplicate prevention, result tallying), testing, GitHub commits, and documentation.

Result / Expected Output

Sample Console Output

```
Enter Voter ID: V1001
Choose Candidate (0-A, 1-B, 2-C): 0
Vote Recorded Successfully
```

```
Enter Voter ID: V1001
Choose Candidate: 1
Duplicate Vote Not Allowed
```

```
Current Statistics:
Candidate A : 42 votes (46.2%)
Candidate B : 35 votes (38.5%)
Candidate C : 18 votes (15.3%)
```

```
Leading Candidate: Candidate A
```



Vote Recorded

Valid voter ID accepted and count updated.



Duplicate Blocked

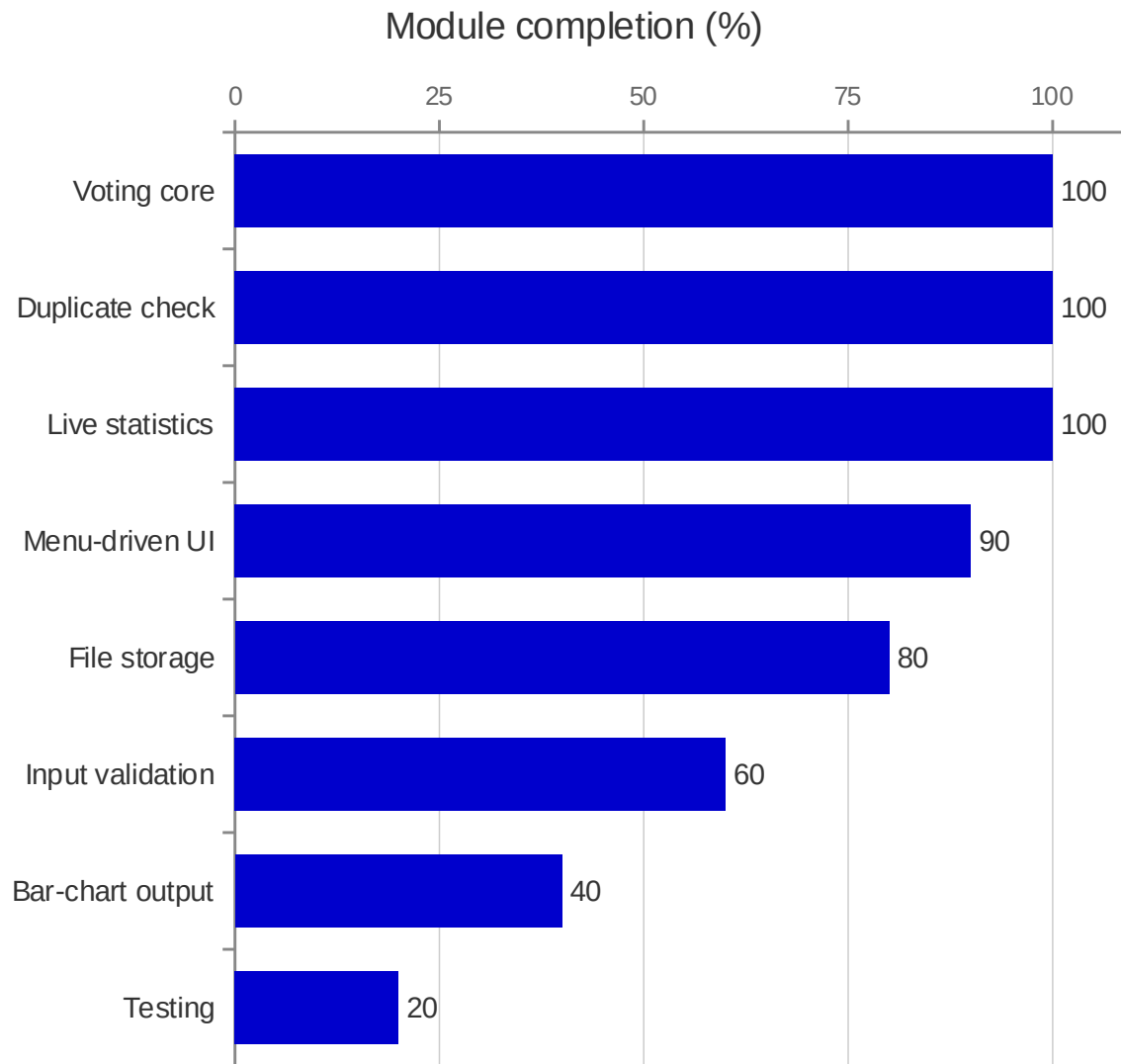
Repeated ID rejected with clear message.



Live Leaderboard

Vote counts, percentages, and leading candidate displayed in real time.

Progress of Project Work (70%)



Overall: 70% Complete

Phase 1 design and core logic are done; Phase 2 added persistence, a menu and validation.

Completed

Vote casting, duplicate prevention, tallying, percentages and leading candidate.

In Progress / Pending

Validation, graph output and test cases make up the remaining 30%.

Phase 2 Enhancements: Python Logic

```
def cast_vote(voter_id, choice):
    """Record a vote if the input is valid and the voter has not voted."""
    if voter_id == "":
        print("Invalid Input: Voter ID cannot be empty")
    elif choice < 0 or choice >= len(candidates):
        print("Invalid Input: Choose a valid candidate number")
    elif voter_id in voted_ids:
        print("Duplicate Vote Not Allowed")
    else:
        votes[choice] += 1
        voted_ids.append(voter_id)
        print("Vote Recorded Successfully")

def show_statistics():
    """Display total votes, per-candidate counts, percentages and leader."""
    total = sum(votes)
    print("\nCurrent Statistics:")
    print("Total Votes:", total)
    for i in range(len(candidates)):
        percent = votes[i] / total * 100 if total > 0 else 0
        print(f"Candidate {candidates[i]} : {votes[i]} votes ({percent:.1f}%)")
    if total > 0:
        top = max(votes)
        leaders = [candidates[i] for i in range(len(votes)) if votes[i] == top]
        if len(leaders) == 1:
            print("\nLeading Candidate: Candidate", leaders[0])
        else:
            print("\nTie between candidates:", ", ".join(leaders))
```

cast_vote()

Validates input, blocks duplicate IDs, then updates the vote count.

show_statistics()

Shows totals, percentages, and the leader or a tie.

Lists

candidates, votes and voted_ids store all data.

Working Output: Menu-Driven System

```
===== ONLINE VOTING SYSTEM =====
1. Cast Vote    2. Show Results
3. Exit
Choice: 1
Enter Voter ID: V1001
Candidate (0-A, 1-B, 2-C): 0
Vote Recorded

Choice: 1
Enter Voter ID: V1001
Candidate (0-A, 1-B, 2-C): 1
Duplicate Vote Not Allowed

Choice: 2
Total Votes: 95
Candidate A: 42 (44.2%)
Candidate B: 35 (36.8%)
Candidate C: 18 (18.9%)
Leading: Candidate A
```

Vote Recorded

A valid new voter ID updates the count and is saved to file.

Duplicate Blocked

A repeated ID is rejected with a clear message.

Live Results

Totals, percentages and the leader are shown on demand.

GitHub Collaboration

Repository & Structure

Repository "Online Polling & Voting System" holds voting.py, votes.json, README.md and a tests folder, with dependencies and usage documented.

Branching & Pull Requests

Each feature (persistence, validation, graph) is built on its own branch and merged to main through a reviewed pull request.

Commit History

Small, meaningful commits, for example:
"Add duplicate voter check"
"Add JSON file persistence"
"Add input validation"

Team Practices

Issues track pending tasks, mentor feedback is applied through PR comments, and README updates go with every milestone.

Remaining Work (30%) & Next Steps

1

Input Validation

Handle non-numeric choices and repeated retries cleanly.

2

Graphical Output

Draw the Votes by Candidate bar chart from live data using matplotlib.

3

Testing

Write test cases for duplicates, invalid input and tie results.

4

Final Documentation

Complete README, screenshots and the final report.

Thank You